A Containerised Compute Cluster for Evaluating the Energy-Aware Runtime (EAR)

Internship Report

College of Computing - Université Mohammed VI Polytechnique (UM6P)

https://github.com/AnasOujja/ear-docker

Author Anas Oujja

Supervisor Prof. Dr. Robert Basmadjian

Professor, College of Computing, UM6P

Head of Toubkal Supercomputer

Period September – November 2025

Acknowledgements

I would like to express my sincere gratitude to **Prof. Dr. Robert Basmadjian** for the opportunity to carry out this internship under his supervision at the College of Computing, Université Mohammed VI Polytechnique. The freedom to explore, encounter failure, and reason through obstacles independently made this a genuinely formative experience.

I also extend my gratitude to the **College of Computing at UM6P** for hosting this internship, and to the **Barcelona Supercomputing Centre (BSC)** and **Lenovo** for developing EAR within their collaboration and making it available to the research community [3].

Abstract

This report documents a three-month internship (September–November 2025) at the College of Computing, UM6P. The objective was to build a simulated HPC cluster environment for testing Energy-Aware Runtime (EAR), a joint BSC–Lenovo energy management framework for supercomputers [1].

EAR's node daemon (EARD) requires root access to real hardware interfaces — specifically the `acpi-cpufreq` kernel driver and Intel RAPL energy counters — on every compute node. This makes testing inside virtual machines impossible and testing on shared production clusters such as Toubkal inadvisable from a security standpoint. The solution was a fully containerised cluster using **Docker**, with compute containers running in privileged mode to grant EARD real hardware access.

All development was carried out on the author's personal laptop (Intel Core i5-1135G7, CentOS Stream 10 on an external hard disk). The setup was later deployed on a university workstation as a portability check. The report covers EAR's architecture, the rationale for the container approach, the complete build process, host-level system configuration, the simulation workflow and what was validated, and a detailed account of the constraints and failures encountered.

Contents

Chapter 1

Context and Motivation

1.1 Toubkal and the Energy Challenge in HPC

As supercomputers scale to tens of thousands of compute nodes, energy consumption becomes a dominant operational cost. The Toubkal supercomputer — the most powerful in Africa, hosted at Université Mohammed VI Polytechnique — faces these pressures directly. Efficient energy management at this scale is both a research priority and an operational necessity.

The Energy-Aware Runtime (EAR), developed jointly by the Barcelona Supercomputing Centre and Lenovo [1], addresses this by providing a transparent, SLURM-integrated framework for energy accounting, monitoring, and optimisation. EAR has been running in production at SuperMUC-NG (LRZ, Munich) since 2019, demonstrating execution time reductions of up to 13% for CPU-bound workloads under its `MIN_TIME_TO_SOLUTION` policy and energy savings of up to 8% for memory-bound workloads under `MIN_ENERGY_TO_SOLUTION` [1]. These results build on earlier work in energy-aware HPC scheduling [6, 7, 8].

1.2 Internship Objective and Choice of EAR Version

The objective was to build a simulated, controllable HPC cluster environment for testing EAR without risk to any production infrastructure. The first step was a thorough analysis of EAR by reading the SC'19 publication [1] and the official BSC documentation [2]. This analysis was essential both to understand EAR's internals and to identify the hardware dependencies that would constrain the simulation design.

EAR 3.2 was chosen as the target version. It is the earliest publicly available release and corresponds directly to the architecture described in the SC'19 paper. Later releases introduced GPU support, a plugin-based policy framework, and an EAR Loader abstraction — none of which are described in the reference publication and all of which would add unnecessary complexity. Using 3.2 keeps the implementation as close as possible to the documented design. The full source code used is available at [4], and the Docker simulation environment produced during this internship is published at [5].

1.3 Development Machine

All development was carried out on the author's personal laptop: an **Intel Core i5-1135G7** (Tiger Lake, 11th Gen, 4 cores / 8 threads, 2.40 GHz base). **CentOS Stream 10** was installed on an external hard disk and booted exclusively for this project, leaving the internal Windows installation intact.

Once the simulation was functional, it was also deployed on a university-provided workstation as a portability check. The two machines were never used together; all development was on the personal laptop.

Two hardware properties have direct technical consequences discussed later in this report. The CPU does support AVX512 (confirmed via `/proc/cpuinfo`), but EAR was compiled with `-disable-avx512` to ensure portability to the university machine which was not supporting

AVX512. Additionally, the choice of CentOS Stream 10 as the host OS introduced a kernel-level incompatibility with the RHEL 8-era container images, which is discussed in detail in Chapter 7.

Chapter 2

EAR 3.2 Architecture

2.1 Overview

EAR delivers three cluster-wide services — energy accounting, monitoring, and optimisation — through five cooperating components [1]. Figure 2.1 shows their deployment layout.

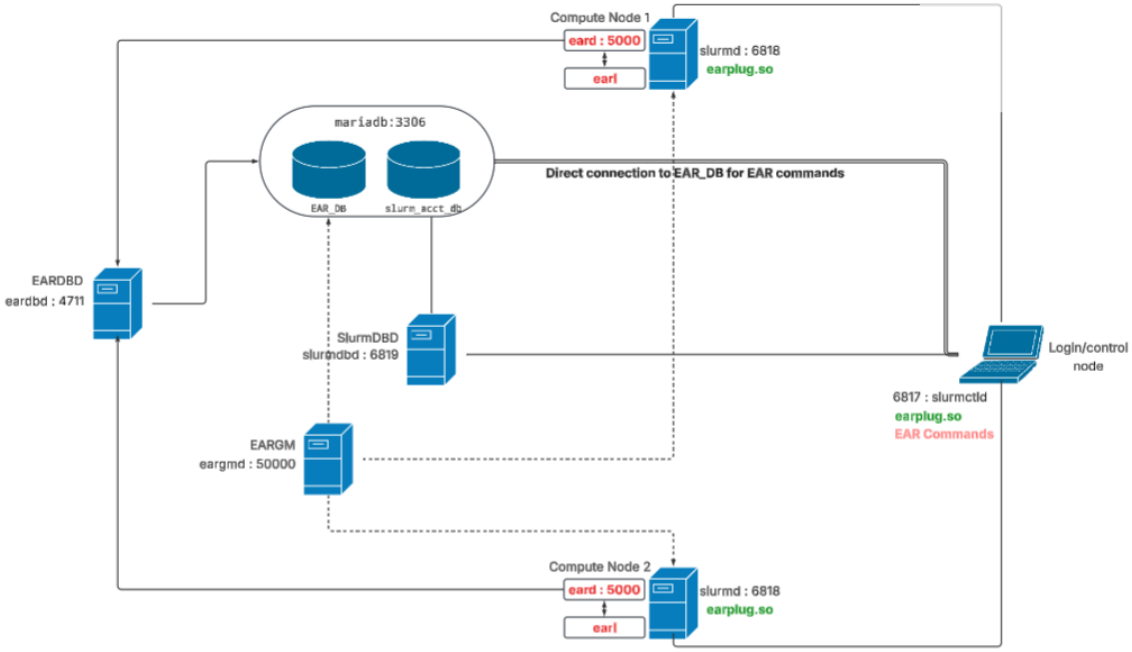

Figure 2.1: *EAR 3.2 deployment architecture. EARD and EARL run on every compute node; EARPLUG is installed cluster-wide on login and compute nodes; EARDBD and EARGM run on separate service nodes; MariaDB provides persistent energy accounting.*

Component	Full Name	Role
EARD	EAR Daemon	Root daemon per compute node; controls DVFS, reads RAPL, serves EARL
EARL	EAR Library	Injected into MPI/MPI+OpenMP jobs; intercepts calls, detects phases, applies energy policies
EARPLUG	EAR SLURM Plug-in	SPANK plug-in; loads EARL at job launch; notifies EARD at job start/end
EARDBD	EAR DB Buffer Daemon	Buffers, aggregates, and replicates energy writes to the database
EARGM	EAR Global Manager	Monitors cluster-wide power; issues warning levels when budgets are exceeded

2.2 EARD — The EAR Daemon

EARD is the most critical component from a simulation standpoint. It runs as a privileged root daemon on every compute node and fulfils three roles [1]: providing EARL with privileged hardware metrics on demand (CPU frequency, memory bandwidth, RAPL energy readings); receiving energy control commands from EARGM; and running an autonomous periodic power monitoring loop that records node-level energy to EARDBD. Root privilege is non-negotiable: EARD must write to the `cpufreq` sysfs interface to change CPU P-states, read RAPL counters from `/sys/class/powercap/intel-rapl/` [11], and access the MSR device at `/dev/msr`. This is the single hardware constraint that determined the entire simulation design.

2.3 EARL — The EAR Library

EARL is a lightweight dynamic library loaded transparently into MPI and hybrid MPI+OpenMP applications via `LD_PRELOAD`, requiring no source modification [1]. Figure 2.2 shows its internal processing pipeline.

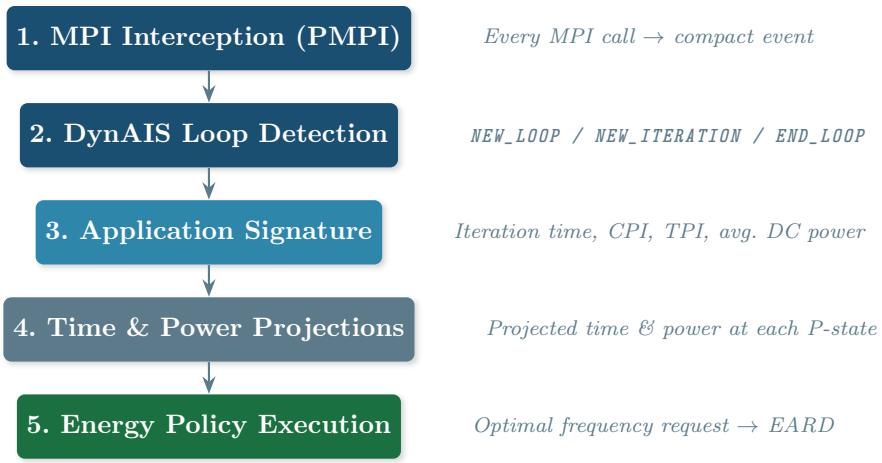

Figure 2.2: *EARL's five internal processing stages [1]. Stages 1–2 run on every MPI call; stages 3–5 run only when DynAIS confirms a stable iterative loop.*

2.3.1 MPI Interception and DynAIS

EARL wraps every MPI function via the PMPI profiling interface. Each call becomes a compact event (call type, source address, destination address) fed into **DynAIS** — EAR's main technical contribution [1]. DynAIS detects iterative computation structures on the fly using a sliding-window comparison over the last N events (`DynAISWindowSize`, default 500). Its multi-level design handles deeply nested loops efficiently, with an average overhead of 0.1 μs per MPI call. It emits three state transitions: `NEW_LOOP`, `NEW_ITERATION`, and `END_LOOP`. The inner comparison loop uses AVX512 SIMD; in this project it was compiled with the AVX2 fallback via `-disable-avx512`.

If no loop is detected within `DynaisTimeout` seconds (default 30), EARL switches to periodic mode and computes a signature every `LibraryPeriod` seconds (default 30).

2.3.2 Application Signature, Projections, and Policies

Once a stable loop is detected, EARL computes the **Application Signature**: four per-iteration metrics — wall-clock iteration time, CPI and TPI from PAPI hardware counters, and average DC node power from RAPL. Combined with the node's pre-computed **System Signature** (six coefficient matrices A–F derived at install time from NAS benchmark runs), EARL projects performance and power at every available P-state:

$$\text{Power}(f) = A \cdot \text{Power}(f_{\text{ref}}) + B \cdot \text{TPI}(f_{\text{ref}}) + C \tag{2.1}$$

$$\text{CPI}(f) = D \cdot \text{CPI}(f_{\text{ref}}) + E \cdot \text{TPI}(f_{\text{ref}}) + F \tag{2.2}$$

$$\text{Time}(f) = \text{Time}(f_{\text{ref}}) \cdot \frac{\text{CPI}(f)}{\text{CPI}(f_{\text{ref}})} \cdot \frac{f_{\text{ref}}}{f} \tag{2.3}$$

Three policies are available. **MONITORING_ONLY** (the default in this project) collects data without changing frequency. **MIN_ENERGY_TO_SOLUTION** steps down from the nominal frequency to the lowest P-state where predicted performance degradation stays within the configured threshold. **MIN_TIME_TO_SOLUTION** starts below nominal and steps up when the projected performance gain justifies the frequency cost.

2.4 EARPLUG, EARGM, and EARDBD

EARPLUG is a SPANK plug-in registered in `plugstack.conf`. It intercepts job submission, sets `LD_PRELOAD` to inject EARL, notifies EARD at job start and end, and forwards EAR policy flags. In this project it is configured as `earlib_default=on`, meaning EARL is automatically loaded for all jobs without any user action.

EARGM monitors aggregate cluster power across two periods ($T_1 = 240\,\text{s}$, $T_2 = 480\,\text{s}$ in this project) and issues warning levels at 85%, 90%, and 95% of the configured energy budget. It runs in passive monitoring mode (`GlobalManagerMode=0`).

EARDBD buffers energy writes from all compute nodes and batch-flushes them to MariaDB at configurable intervals (aggregation every 180 s, insertion every 60 s).

2.5 Hardware Background: DVFS, cpufreq, and RAPL

DVFS (Dynamic Voltage and Frequency Scaling) allows the CPU clock and voltage to be adjusted at runtime. Reducing frequency on a memory-bound application can yield substantial energy savings with minimal performance loss — the physical principle underlying EAR's policies.

Linux exposes DVFS through the **CPUFreq subsystem** at `/sys/devices/system/cpu/cpu*/cpufreq/`. EAR 3.2 requires the `acpi-cpufreq` driver. The default driver on modern Intel systems is `intel_pstate` [9], which operates autonomously and does not expose individual P-state steps to userspace. Switching from `intel_pstate` to `acpi-cpufreq` is a prerequisite for EARD to start.

Intel RAPL [11] provides hardware energy counters per power domain via `/sys/class/powercap/intel-rapl`. The Package domain covers the full CPU socket; PP0 covers processor cores; the DRAM domain covers attached memory but is available only on server-class processors — it is absent on the i5-1135G7 used in this project. Available domains must be probed at runtime; EARD does this during initialisation.

Chapter 3

Simulation Approach

3.1 Why EARD Determines the Approach

All EAR components except EARD can be deployed in any environment without hardware constraints. EARD is the exception: without real root access to the host kernel's hardware interfaces, it cannot function. This eliminates most conventional simulation options.

3.2 Options Considered

Figure 3.1 summarises the decision.

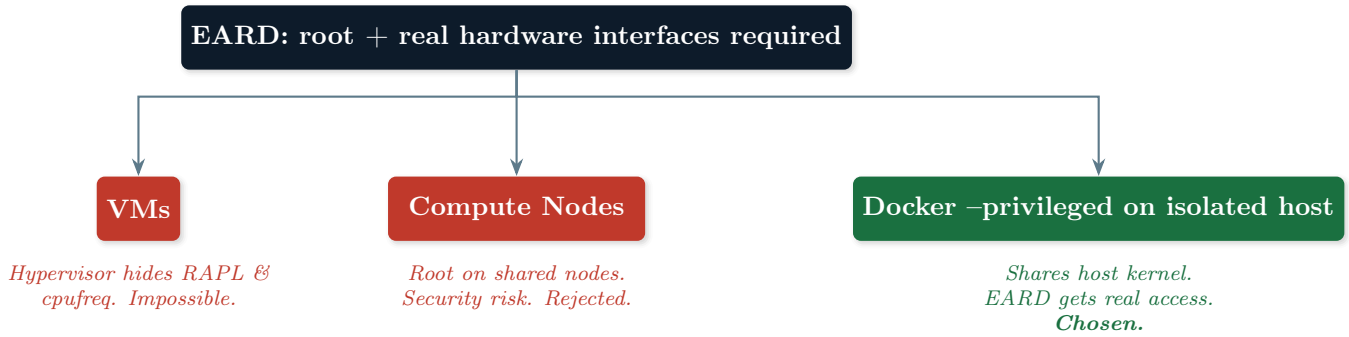

Figure 3.1: *Simulation platform selection tree. EARD's hardware access requirements eliminate VMs; security concerns eliminate shared-cluster options; Docker privileged containers on an isolated host is the only viable approach.*

Virtual Machines: hypervisors present a virtualised CPU to the guest OS. RAPL counters are not exposed, and the `cpufreq` interface inside the guest reflects hypervisor scheduling, not hardware P-states. EARD cannot function inside a VM.

Compute Nodes: Tools like Apptainer could provide hardware access for compute nodes selected to play the role of the simulated compute nodes but would require root capabilities on those machines. The security implications are discussed in Section 3.3.

Docker with privileged containers on an isolated machine: containers share the host kernel. A container started with `privileged: true` has full access to `/sys`, `/dev/msr`, and all hardware interfaces — exactly what EARD requires. Docker Compose handles multi-service orchestration. The trade-off is that all containers share one physical hardware view: RAPL counters reflect aggregate host energy consumption, not per-simulated-node values. The simulation is therefore valid for **functional validation** of EAR's software stack, not for quantitative per-node energy measurement.

3.3 Security: Why Toubkal's Nodes Were Not an Option

Even acting in complete good faith, testing with root on a shared production cluster is incompatible with sound security practice. Information security is conventionally analysed through four properties, and a shared HPC cluster is precisely the kind of environment where

granting an individual user root access threatens all four simultaneously.

Confidentiality is the property that information is not made available or disclosed to unauthorised individuals, entities, or processes. **Integrity** is the property of safeguarding the accuracy and completeness of data and processing methods — ensuring that information is not modified, whether accidentally or maliciously, without authorisation. **Availability** is the property of being accessible and usable upon demand by an authorised entity — a system that cannot deliver its service when needed has failed this property even if confidentiality and integrity remain intact. A fourth and closely related principle is **non-repudiation**: the assurance that an actor cannot later deny having performed a given action, typically enforced through authenticated logging and accountability mechanisms. On a shared cluster, root access undermines non-repudiation as well, because a sufficiently privileged process can alter or erase the very logs that would otherwise attribute an action to a specific user.

The risks of granting root on Toubkal's shared compute nodes map onto each of these properties as follows.

Root on a shared compute node provides unrestricted access to co-running processes' memory, network traffic on the interconnect, and job data on shared filesystems. Multiple users' jobs may be co-located on the same node. No security model can rely on individual restraint as its control mechanism — the risk exists structurally, regardless of intent.

A root process can modify SLURM accounting records, shared configuration, or kernel modules. A misconfigured EARD at root level could unintentionally corrupt shared state or alter kernel parameters affecting every user on the system.

DVFS and RAPL operations can physically destabilise a node — erroneous MSR writes cause machine check exceptions and kernel panics. Beyond the local node, root access on one compute node connected to the internal network creates a privilege escalation path toward service, login, or storage nodes, potentially compromising the entire cluster.

A process with root privileges can, in principle, tamper with or delete the audit trails (SLURM accounting records, system logs) that would otherwise allow administrators to attribute a specific action to a specific user. This weakens accountability for the entire cluster, not just for the node where root access was granted.

Chapter 4

Building the Simulation

This chapter documents the exact build process as derived from the repository at [5].

4.1 Repository Layout

File / Directory	Purpose
`Dockerfile.base`	Common foundation: installs all dependencies, builds SLURM and EAR from source
`Dockerfile.controller`	Login + SLURM controller node
`Dockerfile.node`	Compute node (slurmd + EARD + EARL)
`Dockerfile.slurmdbd`	SLURM database daemon
`Dockerfile.eardbd`	EAR database buffer daemon
`Dockerfile.eargm`	EAR global manager
`docker-compose.yml`	Full cluster orchestration
`.env`	Version pins (SLURM, EAR, image tag)
`build_images.sh`	Host preparation and image build script
`etc-ear/ear.conf`	EAR cluster configuration
`etc-slurm/*.conf`	SLURM configuration files
`db-init/init.sql`	MariaDB initialisation (SLURM database)
`entrypoints/*.sh`	Per-container startup scripts
`munge.key`	Pre-generated munge authentication key
`overrides/hardware_in`	Patched EAR source: topology from `/tmp`
`overrides/energy_node`	Patched EAR source: hardcoded RAPL plugin path
`topo/thread_siblings`	Host CPU topology bitmask (captured pre-build)
`topo/core_siblings`	Host CPU topology bitmask (captured pre-build)
`mpi_test.c`	Simple MPI hello-world for integration testing
`cpu-bound.sh`	CPU-intensive bash workload for SLURM testing
`memory-bound.sh`	Memory-intensive bash workload for SLURM testing

Versions are pinned in `.env`:

.env

```
SLURM_TAG=slurm-21-08-6-1   # SLURM 21.08.6 from SchedMD GitHub
EAR_TAG=3.2                  # EAR 3.2 from eas4dc GitHub
IMAGE_TAG=3.2                # Docker image tag
```

4.2 Host Preparation (build_images.sh)

Before building any Docker image, the host must be prepared. The `build_images.sh` script handles this:

build_images.sh

```
# Copy the munge key from the host (already committed to repo,
# but the script refreshes it from the running munge daemon)
cp /etc/munge/munge.key ./munge.key

# Capture host CPU topology bitmasks needed by the hardware_info override
cp /sys/devices/system/cpu/cpu0/topology/thread_siblings ./topo/thread_siblings
cp /sys/devices/system/cpu/cpu0/topology/core_siblings ./topo/core_siblings

# Build the shared base image first, then all role images
docker compose build base
docker compose build
```

The munge key provides cryptographic authentication for all SLURM inter-service communications. All containers must share the same key, so it is baked into the base image. The topology bitmasks are required by the `hardware_info.c` source override (see Section 4.3).

4.3 The Base Image (Dockerfile.base)

The base image is the most complex and time-consuming part of the build. It installs all shared system dependencies, then builds both SLURM and EAR from source so that they are compiled against each other's libraries in a consistent environment.

System Dependencies

System packages installed in Dockerfile.base

```
yum -y install \
    mariadb mariadb-devel \      # EAR and SLURM database connectivity
    munge munge-devel \          # SLURM authentication
    papi papi-devel \            # Hardware performance counters (CPI, TPI)
    gsl gsl-devel \              # System signature matrix computation
    hwloc hwloc-devel \          # Hardware locality (used by SLURM and EAR)
    libevent libevent-devel \    # Async I/O (PMIx dependency)
    openmpi openmpi-devel        # MPI library
```

SLURM Build

SLURM 21.08.6 is cloned from SchedMD's GitHub and built from source. Building from source is essential: it ensures SLURM is compiled with support for the exact libraries present in the image (munge, MariaDB) and makes the SLURM headers available for EAR's compilation step.

SLURM build in `Dockerfile.base`

```
git clone -b slurm-21-08-6-1 --single-branch --depth=1 \
    https://github.com/SchedMD/slurm.git
cd slurm
./configure --enable-debug --prefix=/usr \
    --sysconfdir=/etc/slurm \
    --with-mysql_config=/usr/bin \
    --libdir=/usr/lib64
make -j$(nproc) install
```

EAR Source Patches

Before EAR is compiled, two source files are replaced with patched versions from the `overrides/` directory.

`hardware_info.c`: EAR 3.2's original implementation reads CPU topology from sysfs paths under `/sys/devices/system/cpu/cpu0/topology/`. In principle these paths should have been readable from inside the compute containers: the containers run in privileged mode, and the sysfs paths EARD reads from were verified to be correct and present. In practice, EARD's topology initialisation did not work correctly when reading directly from these sysfs paths inside the container — the exact reason was not identified despite the paths, permissions, and content all appearing correct from manual inspection. As a practical workaround, the relevant bitmasks were instead captured once on the host at build time and copied into the image at `/tmp/thread_siblings` and `/tmp/core_siblings`, and `hardware_info.c` was patched to read from these fixed files instead of the live sysfs path:

Key change in `overrides/hardware_info.c`

```
/* Original: reads live from sysfs -- did not work correctly
    in-container */
/* Patched: reads from /tmp -- bitmasks captured on host at build
    time   */
if (file_is_accessible("/tmp/thread_siblings"))
    hardware_sibling_read("/tmp/thread_siblings", &topo->threads);
if (file_is_accessible("/tmp/core_siblings") && topo->threads > 0) {
    hardware_sibling_read("/tmp/core_siblings", &topo->cores);
    topo->cores /= topo->threads;
}
```

On the development machine (i5-1135G7): `thread_siblings = 0x11` (2 threads), `core_siblings = 0xff` (4 cores per socket × 2 threads = 8 logical CPUs). This matches the actual hardware and is what EARD uses to initialise its CPU management.

`energy_node.c`: EAR's energy measurement layer resolves the path to its RAPL plugin from configuration at runtime. As with the topology files, this resolution did not work reliably inside the compute containers — EARD intermittently failed to load the RAPL energy plugin even though the plugin file and the configured path both appeared correct. The precise cause

was not identified. As a practical workaround, the path was hardcoded directly in the source rather than left to be resolved at runtime:

Key change in `overrides/energy_node.c`

```
/* Original: resolves path from ear.conf configuration */
/* Patched: hardcoded path to ensure RAPL plugin is always found */
sprintf(energy_objc, "%s/energy/%s",
        "/usr/lib/plugins", "energy_rapl.so");
```

This ensures EARD always uses the Intel RAPL plugin for energy measurement, regardless of how the plugin path is configured in `ear.conf`, which is consistent with the `PluginEnergy=energy_rapl` setting in `ear.conf`.

EAR Build

After applying the patches, EAR 3.2 is built:

EAR 3.2 build in `Dockerfile.base`

```
CC=gcc MPICC=/usr/lib64/openmpi/bin/mpicc \
USER=ear GROUP=ear \
./configure --disable-avx512 \     # Use AVX2 fallback for portability
    --prefix=/usr \
    --sysconfdir=/etc/ear \
    --localstatedir=/var/ear \
    --with-papi=/usr \             # Hardware performance counters
    --with-mysql=/usr \            # Database connectivity
    --with-slurm=/usr              # SLURM headers (built in previous
        step)
make && make etc.install
```

The `-disable-avx512` flag selects the AVX2 SIMD backend for DynAIS. Although the development laptop supports AVX512, this flag was used to guarantee compatibility with the university workstation. Both `make` targets must run: `make` builds all binaries, and `make etc.install` installs the default configuration templates.

4.4 Role Images

All role images use a two-stage build: they inherit from `slurm-ear-base` and install only the EAR component specific to their role using the corresponding `make` target on the EAR source tree left in `/tmp/` by the base image:

Image	Make target(s)	Key installed binaries
Dockerfile.controller	earplug.install commands.install	/usr/lib/earplug.so, econtrol, eacct, ereport
Dockerfile.node	install eard.install earl.install	/usr/sbin/eard, /usr/lib/libear.so, /usr/bin/tools/coeffs_null
Dockerfile.slurmdbd	(none — SLURM already installed)	/usr/sbin/slurmdbd
Dockerfile.eardbd	eardbd.install	/usr/sbin/eardbd
Dockerfile.eargm	eargmd.install	/usr/sbin/eargmd

4.5 Configuration Files

ear.conf

The master EAR configuration, shared across all containers via the etc_ear Docker volume. Key settings:

Key fields in etc-ear/ear.conf

```
PluginEnergy=energy_rapl            # Use Intel RAPL for energy
    measurement

MariaDBHost=mysql    MariaDBPort=3306    MariaDBDatabase=EAR_DB
MariaDBUser=ear_db   MariaDBPassw=earpass
MariaDBCommandsUser=ear_commands    MariaDBCommandsPassw=earpass

GlobalManagerHost=eargm    GlobalManagerPort=50000
GlobalManagerMode=0        # Passive monitoring
GlobalManagerPeriodT1=240 GlobalManagerPeriodT2=480
GlobalManagerWarningsPerc=85,90,95

NodeDaemonPort=5000       NodeUseEARDBD=1    NodeForceFrequencies=1
DBDaemonPortTCP=4711      DBDaemonPortSecTCP=4712
    DBDaemonSyncPort=4713

DefaultPowerPolicy=monitoring      # Default: collect data only
Policy=min_time Settings=0.7 DefaultFreq=2.0
Policy=min_energy Settings=0.1 DefaultFreq=2.3  Privileged=1

TmpDir=/var/ear    EtcDir=/etc/ear    InstDir=/usr
CoefficientsDir=/etc/ear/coeffs

Island=0 Nodes=c[1-2] DBIP=eardbd DBSECIP=eardbd \
    min_power=0 max_power=800 error_power=1000 max_temp=120
```

slurm.conf

Node names must match Docker container hostnames exactly, since Docker Compose's internal DNS resolves hostnames by container name. CPUs are set to 2 per node matching the cpuset assignment in the Compose file:

Node and partition definition in etc-slurm/slurm.conf

```
ClusterName=linux    ControlMachine=slurmctld
MpiDefault=none      ProctrackType=proctrack/linuxproc

NodeName=c1 CPUs=2 Boards=1 SocketsPerBoard=1 \
    CoresPerSocket=1 ThreadsPerCore=2 RealMemory=7462
NodeName=c2 CPUs=2 Boards=1 SocketsPerBoard=1 \
    CoresPerSocket=1 ThreadsPerCore=2 RealMemory=7462

PartitionName=normal Default=yes Nodes=c[1-2] State=UP
```

plugstack.conf and the EAR SPANK plug-in

EARPLUG is registered with SLURM via `plugstack.conf`. The `earlib_default=on` flag means EARL is loaded automatically into every job without any user action:

etc-slurm/plugstack.conf

```
required /usr/lib/earplug.so \
    prefix=/usr  sysconfdir=/etc/ear  localstatedir=/var/ear \
    earlib_default=on
```

Database initialisation

The `db-init/init.sql` file is mounted as a MariaDB init script and runs automatically on first container startup. It creates only the SLURM accounting database:

db-init/init.sql

```
CREATE DATABASE IF NOT EXISTS slurm_acct_db;
CREATE USER IF NOT EXISTS 'slurm'@'%' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON slurm_acct_db.* TO 'slurm'@'%';
FLUSH PRIVILEGES;
```

The `EAR_DB` database itself is **not** created automatically by EARDBD. It is created explicitly, on first startup, by running EAR's provided `edb_create` command-line tool, which connects to MariaDB using the credentials in `ear.conf` and creates the `EAR_DB` schema along with the `ear_db` and `ear_commands` users. This step must be run once, after MariaDB is up but before EARDBD or EARD attempt to write any accounting data; the recommended place to run it is from within the `eardbd` container's entrypoint, guarded so that it only runs on first startup.

4.6 Docker Compose Topology

Figure 4.1 shows the complete topology as defined in `docker-compose.yml`.

Key design decisions in the Compose file:

Both `c1` and `c2` run with `privileged: true`, giving EARD unrestricted access to `/sys` and `/dev/msr` inside each container. Each service is pinned to dedicated host CPU cores via the `cpuset` directive, in preparation for the multi-socket architecture described in Section 5. Each EAR daemon gets its own log volume to avoid log file conflicts between containers sharing the same image. All service containers are on a shared bridge network where container names

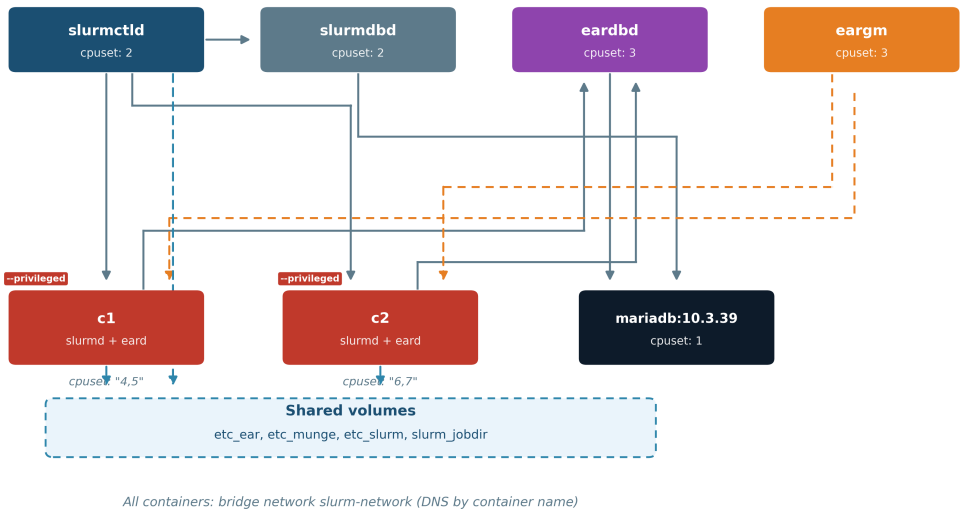

Figure 4.1: *Docker Compose cluster topology. Both compute nodes run with* `privileged: true` *and are pinned to dedicated CPU cores. Each EAR daemon gets its own log volume (*`var_log_ear_c1`*,* `var_log_ear_c2`*,* `var_log_ear_dbd`*,* `var_log_ear_gm`*) mounted at* `/var/ear`*.*

serve as DNS hostnames — this is why node names in `slurm.conf` must match exactly.

4.7 Container Startup Sequence

The `common-wait.sh` helper provides a `wait_for_tcp(host, port)` function used by every entrypoint to enforce startup ordering without relying on Docker's health check system. Figure 4.2 shows the resulting startup dependency graph: a directed acyclic graph (DAG) in which each edge indicates that the source container must be ready before the destination container begins its own startup sequence.

The compute node entrypoint (`entrypoint-node.sh`) deserves particular attention:

entrypoints/entrypoint-node.sh

```
gosu munge /usr/sbin/munged              # Start MUNGE authentication

wait_for_tcp slurmctld 6817 "slurmctld"
wait_for_tcp eardbd    4711 "eardbd"

# Generate dummy coefficients (bypass the real learning phase)
# Range: max_freq=2400000 kHz, min_freq=400000 kHz
cd /etc/ear/coeffs/
/usr/bin/tools/coeffs_null default 2400000 400000

/usr/sbin/slurmd       # Start SLURM compute daemon (background)
/usr/sbin/eard         # Start EARD (requires acpi-cpufreq on host)
```

The `coeffs_null` tool generates zero-valued coefficient files, bypassing the full system signature learning phase (which would require running NAS benchmarks at every P-state). This means EARL cannot make meaningful energy projections, but it allows the simulation to start without the full learning infrastructure. The coefficients cover the frequency range 400 MHz to 2.4 GHz, matching the i5-1135G7's available P-states.

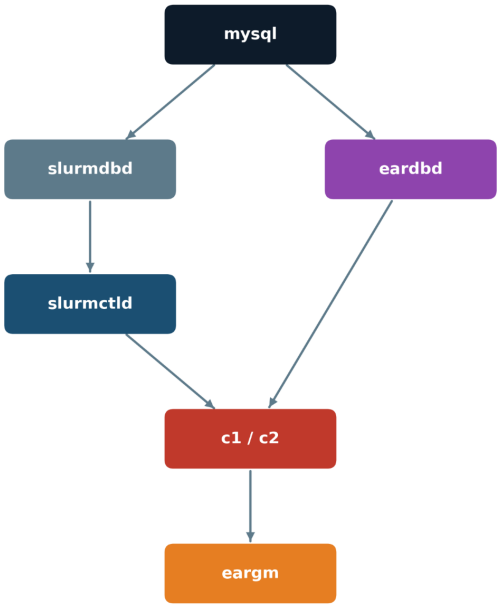

Figure 4.2: *Container startup dependency graph. Each edge represents a "must be ready before" relationship enforced by* `wait_for_tcp` *in the corresponding entrypoint script.* `mysql` *is the root of the graph;* `eargm` *is the final container to start, since it depends transitively on every other service.*

Chapter 5

System Configuration

5.1 Switching the CPU Scaling Driver

EAR 3.2's internal frequency management checks that the active cpufreq driver is `acpi-cpufreq` and aborts on startup if it is not. On modern Intel systems the default is `intel_pstate` [9], which does not expose individual P-state steps to userspace. The switch must be made before EARD can start.

On CentOS/RHEL systems, the recommended method is `grubby`:

Switching to `acpi-cpufreq` on CentOS Stream 10

```
# Add the kernel parameter permanently to all boot entries
sudo grubby --update-kernel=ALL --args="intel_pstate=disable"

# Reboot and verify
sudo reboot
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_driver
# Expected: acpi-cpufreq
```

This modifies the GRUB boot configuration. On the personal laptop used for this project, this change — combined with the sustained DVFS operations performed by EARD and the CPU isolation scheme described in the next section — was a contributing factor to hardware instability leading to eventual system failure, as described in Chapter 7.

5.2 CPU Pinning

CPU pinning was introduced with a specific architectural goal in mind, not merely to reduce scheduling noise. The fundamental limitation of the container simulation is that all containers share one physical hardware view: RAPL energy counters are not virtualised, so any reading taken inside a compute container reflects the energy of the entire host rather than the individual "node" that container is supposed to represent. On a **multi-socket machine**, however, Intel RAPL exposes one Package domain per physical socket, each readable independently. The plan was therefore the following: if a multi-socket workstation became available during the internship, each compute container would be pinned exclusively to the CPUs belonging to one socket, and with some targeted source-level adjustments to EARD's RAPL initialisation, each container could be made to read and act on only its corresponding socket's energy counters. This would provide a genuine per-node energy isolation within a single host, making the simulation substantially more realistic.

CPU pinning via `cpuset` was put in place in anticipation of this architecture. Unfortunately, neither the personal laptop nor the university-provided workstation was a multi-socket machine — both have a single physical CPU — so the per-socket isolation was never exercised in practice. The pinning was kept in the configuration anyway, as it still provides the sec-

ondary benefit of reducing scheduling interference between containers and bounding EARD's frequency changes to specific cores.

At the kernel level, `isolcpus` removes CPUs from the general scheduler [10]. On the development machine, only **CPU 0 was left for the kernel and general system tasks**; CPUs 1 through 7 were isolated and reserved for the containers:

Combining driver switch and CPU isolation

```
# Disable intel_pstate AND isolate CPUs 1-7, leaving only CPU 0
# available for the kernel and ordinary system processes
sudo grubby --update-kernel=ALL \
    --args="intel_pstate=disable isolcpus=1-7"
sudo reboot
```

For dynamic management, the `cset` tool (package `cpuset`) wraps the cgroup cpuset interface:

Dynamic CPU shielding with `cset`

```
sudo dnf install cpuset
# Shield CPUs 1-7 from general scheduling, leaving CPU 0 for the
    kernel
sudo cset shield --cpu 1-7 --kthread=on
```

At the Docker level, the `cpuset` directive in `docker-compose.yml` locks each container to its designated cores (as shown in Figure 4.1).

Reserving a single logical CPU for the kernel and all general system tasks on a 4-core/8-thread laptop processor is an aggressive configuration. In hindsight, this is a plausible contributing factor to the hardware instability discussed in Section 7.5. If CPU isolation is applied in future work, More than 1 logical CPUs should be reserved for the kernel.

Chapter 6

Running the Simulation

6.1 Prerequisites Checklist

Before starting the simulation, verify:

1. **acpi-cpufreq is active:** `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_driver` must return `acpi-cpufreq`.
2. **Docker is installed.**
3. **Munge is installed and munge key is generated** (Do it manually or uncomment the corresponding command lines in the images building script).
4. **The host has sufficient CPU cores** (More than 8 logical cores recommended so you have more flexibility in terms of CPU pining).
5. **Internet access** is available during the build step (SLURM and EAR are downloaded from GitHub).

6.2 Build and Start

Complete build and startup from a fresh clone

```
# 1. Clone the repository
git clone https://github.com/AnasOujja/ear-docker.git
cd ear-docker

# 2. Prepare the host and build all images
#    (requires munge installed and running on the host)
chmod +x build_images.sh
./build_images.sh
# This refreshes munge.key, captures CPU topology,
# builds the base image, then builds all role images.
# Expect 15-30 minutes depending on network and hardware.

# 3. Start the full cluster
docker compose up -d

# 4. Create the EAR database (first run only)
docker exec -it eardbd edb_create

# 5. Monitor startup progress
docker compose logs -f
```

6.3 Verifying the Cluster

Cluster health verification

```
# Check SLURM node states from inside the controller
docker exec -it slurmctld bash
sinfo
# Both c1 and c2 should show as 'idle'

# Verify RAPL is accessible from c1 (privileged container)
docker exec -it c1 bash
cat /sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj
# A non-zero value confirms real RAPL access

# Confirm the cpufreq driver inside the compute container
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_driver
# Must return: acpi-cpufreq

# Check EARD is running
ps aux | grep eard

# Check EARDBD connection to database
docker compose logs eardbd | grep "ready"
```

6.4 Submitting Jobs and Test Workloads

The repository includes three test workloads. Since `earlib_default=on`, EARL is automatically loaded for all jobs:

Job submission and test workloads

```
# From inside the slurmctld container:

# Simple validation job
srun -N 2 sleep 30

# CPU-bound bash workload (cpu-bound.sh)
srun -N 2 /data/cpu-bound.sh 1000000

# Memory-bound bash workload (memory-bound.sh, requires /dev/shm)
srun -N 2 /data/memory-bound.sh 512

# MPI hello-world (mpi_test.c -- requires MPI+SLURM integration)
# Compile:
mpicc -o /data/mpi_test /data/mpi_test.c
# Run:
srun -N 2 -n 4 --mpi=pmix /data/mpi_test

# Query EAR accounting data
eacct -j <jobid>
ereport -n c1
```

Chapter 7

Constraints and Lessons Learned

7.1 What Was Validated

The following were confirmed to work end-to-end:

SLURM job scheduling was fully operational: both `c1` and `c2` registered as idle nodes, and bash-script jobs (`srun sleep X`, `srun hostname`, the provided `cpu-bound.sh` and `memory-bound.sh` scripts) were submitted, executed, and completed correctly. SLURM accounting data was recorded in the `slurm_acct_db` database. RAPL energy counters were confirmed readable from inside the privileged compute containers. The `acpi-cpufreq` driver was active and accessible to EARD. The `EAR_DB` database was successfully created via `edb_create`. The complete build pipeline — including SLURM and EAR compilation from source with the applied source patches — was validated on both the personal laptop and the university workstation.

7.2 MPI and SLURM Integration Failure

Running MPI jobs through SLURM with EARL engaged via EARPLUG failed. The failure occurs before any application code executes, and therefore before EARL has any opportunity to intercept MPI calls — meaning the problem lies in how SLURM launches and manages MPI processes, not in EAR itself.

The root cause was not identified. Multiple version combinations were tested, including those recommended in EAR's official documentation: different releases of OpenMPI with PMI2 and PMIx support compiled in or out, different SLURM minor versions, and various `-mpi` flags passed to `srun`. None produced a working MPI job. In each case the job either exited immediately or SLURM failed to manage the MPI process lifecycle, in either case without actionable diagnostic output.

The only interpretation that cannot be ruled out at this point is a structural one: running MPI through SLURM inside Docker containers may require the host OS and the container base image to share compatible system library generations — particularly around the process management interface used between SLURM and the MPI runtime. A mismatch between those generations could make MPI through SLURM unreliable in this setup regardless of which specific versions are chosen inside the container. If this interpretation is correct, the problem cannot be solved by tuning versions within the current environment, but would require a host OS that is compatible with the container base. However, this too has not been confirmed, and it should be treated as a hypothesis to test rather than a diagnosis.

The practical consequence is that EARL's application signature computation, DynAIS phase detection, and energy policy enforcement were never exercised. Only bash-script jobs, which bypass the MPI stack entirely, were validated.

7.3 Dummy Coefficients and the Learning Phase

The compute node entrypoint generates zero-valued system signature coefficients via `coeffs_null`. This bypasses the real learning phase, which requires running NAS benchmark kernels at every available P-state under EARD supervision. The implication is that even if MPI jobs had been running, EARL could not have made meaningful energy projections from these coefficients. For a genuine energy policy test, the learning phase must be completed first.

7.4 Shared Hardware View

All containers share the host's physical hardware. RAPL counters read from inside any container reflect aggregate host energy consumption, not per-simulated-node values. Similarly, a CPU frequency change applied by one container's EARD instance affects all cores used by all other containers. The simulation is therefore valid for functional validation of EAR's software stack, but cannot produce quantitative per-node energy measurements on a single-socket machine.

This limitation is not entirely unavoidable. As described in Section 5, the CPU pinning infrastructure was introduced with a multi-socket architecture in mind: on a dual-socket workstation, each compute container could be pinned to one socket, and with targeted adjustments to EARD's RAPL initialisation, each container could read and act on only its socket's energy counters. Neither machine available during the internship was multi-socket, so this path was not exercised, but the groundwork for it is already in place.

7.5 Hardware Damage to the Development Machine

This is the most important practical lesson of the internship: **do not run this simulation on a personal machine you cannot afford to lose.**

During the internship, the development laptop began experiencing frequent unexpected reboots and system freezes. Critically, these occurred not only during EAR simulation sessions but also during normal Windows usage on the internal disk, making clear that the issue was hardware-level rather than OS-specific.

The situation worsened progressively over two months, even after the GRUB tweaks (`isolcpus`, `intel_pstate=disable`) were removed, demonstrating that the damage was not purely a software configuration issue. It culminated in the boot failure shown in Figure 7.1: ACPI BIOS errors (`Could not resolve symbol [\_SB.PCI0]`, `AE_NOT_FOUND`) appeared on every boot attempt, the system dropped into emergency mode.

The most likely contributing factor is the combination of repeated P-state changes applied by EARD when tested in `MONITORING_ONLY` mode, the non-default `acpi-cpufreq` driver on hardware designed to be managed by `intel_pstate`, and the aggressive CPU isolation scheme that left only a single logical CPU available to the kernel (Section 5). A kernel starved of CPU time under sustained interrupt and scheduling pressure, combined with active low-level frequency manipulation, is a plausible path to the kind of firmware-level instability observed. Laptop CPUs are also not designed or validated for this kind of continuous programmatic frequency manipulation in the first place.

7.6 Path Forward

In priority order, the following steps would advance the project to a fully functional state:

1. **Use a host running RockyLinux 8 or AlmaLinux 8**: this eliminates the PMI version

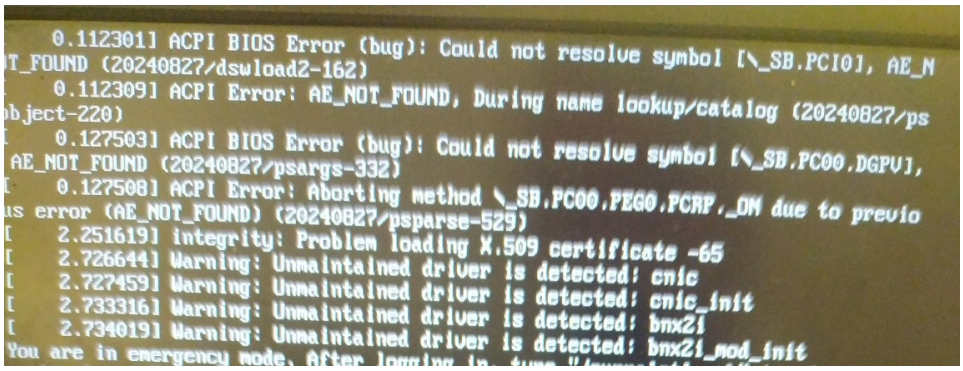

Figure 7.1: *Boot failure encountered after sustained system tweaking. ACPI BIOS errors indicate the firmware cannot resolve hardware symbols. Unmaintained driver warnings (`cnic`, `bnx2i`) are logged, and the system drops into emergency mode. This occurred after two months of instability and persisted even after reverting the GRUB changes.*

mismatch with the RHEL 8 container images.

2. Resolve MPI+SLURM: with a matching host OS, retest MPI job submission.

3. Reserve more than one CPU for the kernel: when applying `isolcpus`, leave two or more logical CPUs for the kernel and system tasks.

4. Run the real learning phase: once MPI is operational, execute the NAS benchmark suite at each P-state to compute real system signature coefficients, replacing the dummy ones generated by `coeffs_null`.

5. Validate EARL: with MPI jobs running and real coefficients in place, submit workloads using the provided `mpi_test.c` and the EAR policy flags, then query results with `eacct` and `ereport`.

6. Pursue per-node energy isolation on a multi-socket machine: the shared hardware view is the deepest limitation of the current simulation. On a multi-socket workstation, Intel RAPL exposes one independent Package domain per physical socket. By pinning each compute container exclusively to the CPUs of one socket and making targeted adjustments to EARD's RAPL initialisation — directing it to read only the energy counter of its assigned socket — it would be possible to achieve genuine per-node energy isolation within a single host. The CPU pinning infrastructure already present in `docker-compose.yml` and the `isolcpus` configuration was put in place with precisely this architecture in mind. Neither the personal laptop nor the university workstation were multi-socket machines, so this path was not pursued during the internship, but it represents a meaningful next step toward a quantitatively valid simulation that does not require a full multi-node cluster.

Chapter 8

Conclusion

This internship produced a complete, buildable containerised simulation of an EAR 3.2-enabled HPC cluster. The repository at [5] contains everything needed to reproduce the environment: six Dockerfiles, a full Docker Compose orchestration, all configuration files, per-container startup scripts, two source code patches, and test workloads.

The **analysis phase** established that EAR 3.2's hardware requirements — the `acpi-cpufreq` driver and RAPL access — make virtual machine simulation impossible and shared-cluster testing a security risk. These are not incidental constraints but fundamental design facts of EAR that determine the entire simulation strategy.

The **build phase** produced a working containerised SLURM cluster with all five EAR components built from source, a MariaDB database initialised via `edb_create`, and the two source code patches necessary to make EAR function correctly in a container environment. SLURM job scheduling, EARD hardware access, EARDBD buffering, and database persistence were all validated end-to-end with bash-script jobs.

The **unresolved problem** is the MPI and SLURM integration failure, which prevented EARL from being exercised. The potential root cause is a PMI interface version mismatch between the RHEL 8 container images and the CentOS Stream 10 host kernel. The correct fix is not to change the container base but to run the simulation on a host OS in the RHEL 8 family, where kernel and system libraries are version-consistent with the containers.

The most important lesson that only experience could teach is that low-level CPU power management manipulation on a personal laptop is a real hardware risk, and that this risk is compounded by aggressive CPU isolation that leaves the kernel with too little room to operate. The project should be continued on a dedicated, expendable server-class machine with a matching host OS and a more conservative CPU isolation scheme.

Bibliography

[1] J. Corbalán and L. Brochard, "EAR: Energy management framework for supercomputers," in *Proc. SC '19 Workshops*, Denver, CO, November 2019.

[2] Barcelona Supercomputing Centre and Lenovo, "Energy Aware Runtime (EAR) User Guide," 2017. https://www.bsc.es/sites/default/files/public/bscw2/content/software-app/technical-documentation/ear_guide.pdf

[3] Barcelona Supercomputing Centre, "EAR: Energy management framework for HPC." https://www.bsc.es/research-and-development/software-and-apps/software-list/ear-energy-management-framework-hpc

[4] EAR Development Team, "EAR: Energy Aware Runtime." https://github.com/eas4dc/EAR

[5] A. Oujja, "ear-docker: Containerised EAR simulation environment." https://github.com/AnasOujja/ear-docker

[6] R. Bell, L. Brochard, D. DeSotta, R. Panda, and F. Thomas, "Energy-Aware job scheduling for cluster environments," US Patent 8,527,997 B2, 2011.

[7] L. Brochard, R. Panda, D. DeSota, F. Thomas, and R. H. Bell Jr., "Power and energy-aware processor scheduling," *ACM SIGSOFT Software Engineering Notes*, vol. 36, no. 5, 2011.

[8] A. Auweter et al., "A case study of energy aware scheduling on SuperMUC," in *International Supercomputing Conference*, Springer, Cham, 2014.

[9] Linux Kernel Documentation, "intel_pstate CPU Performance Scaling Driver." https://docs.kernel.org/admin-guide/pm/intel_pstate.html

[10] Linux Kernel Documentation, "CPUSETS." https://docs.kernel.org/admin-guide/cgroup-v1/cpusets.html

[11] V. Weaver, "RAPL Energy Measurements," University of Maine. https://web.eece.maine.edu/~vweaver/projects/rapl/